

# Axela Brockett UE5 Plugin: Utility AI

## v1.1.0 Documentation

GitHub download link: <https://github.com/AxelaBrockett/Utility-AI-Plugin-UE5.6>

### Contents:

|                                                 |    |
|-------------------------------------------------|----|
| Basic Overview of the Plugin.....               | 3  |
| FAxelaBrockettUtilityAIBaseModule.....          | 4  |
| FAxelaBrockettUtilityAIEditorBaseModule.....    | 5  |
| UABUtilityComponent.....                        | 6  |
| UABUtilityActionBase.....                       | 12 |
| UABUtilityRequirementBase.....                  | 18 |
| UABUtilityFactorBase.....                       | 20 |
| UABUtilityActionBlueprintBase.....              | 22 |
| UABUtilityRequirementBlueprintBase.....         | 25 |
| UABUtilityFactorBlueprintBase.....              | 27 |
| UABUtilityDataAsset.....                        | 29 |
| UABUtilitySettings.....                         | 30 |
| UABUtilitySubsystem.....                        | 31 |
| FGameplayDebuggerCategory_Utility.....          | 33 |
| UABUtilityDataAssetValidator.....               | 34 |
| ABUtilityTypes.h.....                           | 35 |
| ABUtilityHelpers.h.....                         | 36 |
| Examples.....                                   | 38 |
| Component Setup.....                            | 38 |
| Code Action.....                                | 39 |
| Blueprint Action.....                           | 40 |
| Action Setup in Data Asset.....                 | 40 |
| Code Requirement.....                           | 40 |
| Blueprint Requirement.....                      | 41 |
| Multi-Requirement.....                          | 41 |
| Code Factor.....                                | 41 |
| Blueprint Factor.....                           | 42 |
| Requirement and Factor Setup in Data Asset..... | 42 |
| Asynchronous Logic and Parallel For.....        | 42 |
| Validation.....                                 | 43 |
| Debugging.....                                  | 43 |
| Comments.....                                   | 44 |



## Basic Overview of the Plugin

Utility AI is a decision making method for AI that gives the AI a list of Actions to choose from, gives each of those Actions a score, and then the AI picks the most appropriate Action, most commonly the highest scoring one. This plugin uses the following terms throughout:

- Action
  - A task the AI can do, similar to a Behavior Tree task or State Tree task
- Requirement
  - A prerequisite that must be passed for the action to be available in the moment
- Factor
  - A Factor is considered when calculating the score for an Action

Other terms will also be explained as they appear within the documentation.

The plugin contains a Utility Component, base classes for an Action, a Requirement and a Factor, a Data Asset class, helper files, debugging and validation code. This documentation will go over each of the available classes and how to use each of them to create a usable AI.

## FAxelaBrockettUtilityAIBaseModule

This is the base module class that is used for the runtime part of this plugin.

**Parent class:** *IModuleInterface*

**Functions:**

| Function                                      | Params |
|-----------------------------------------------|--------|
| <b><i>IModuleInterface implementation</i></b> |        |
| <i>virtual void StartupModule() override</i>  |        |
| <i>virtual void ShutdownModule() override</i> |        |

## FAxelaBrockettUtilityAIEditorBaseModule

This is the base module class that is used for the editor part of this plugin.

**Parent class:** *IModuleInterface*

**Functions:**

| Function                                      | Params |
|-----------------------------------------------|--------|
| <b><i>IModuleInterface implementation</i></b> |        |
| <i>virtual void StartupModule() override</i>  |        |
| <i>virtual void ShutdownModule() override</i> |        |

## UABUtilityComponent

This is the Brain Component that is run by the AI Controller. It contains a soft pointer (unloaded) to a Utility Data Asset that has the available Actions, and has both synchronous and asynchronous logic available to the user. The component is completely event driven, so ticking is actively turned off in the constructor, and the *SetComponentTickEnabled(bool bEnabled)* function has been overridden to ensure tick does not get activated. It is not recommended to allow this component to tick.

**Parent class:** *UBrainComponent*

### Functions:

| Function                                                                                                                 | Params                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <i>UABUtilityComponent()</i>                                                                                             |                                                                                         |
| <b><i>UBrainComponent overrides</i></b>                                                                                  |                                                                                         |
| <i>void SetComponentTickEnabled(bool bEnabled) override</i>                                                              | <i>bEnabled</i> - Usually allows users to activate or deactivate the tick functionality |
| <i>void StartLogic() override</i>                                                                                        |                                                                                         |
| <i>void PauseLogic(const FString&amp; Reason) override</i>                                                               | <i>Reason</i> - Allows users to specify why the component is pausing                    |
| <i>EAllLogicResuming::Type ResumeLogic(const FString&amp; Reason) override</i>                                           | <i>Reason</i> - Allows users to specify why the component is now resuming               |
| <i>void RestartLogic() override</i>                                                                                      |                                                                                         |
| <i>void StopLogic(const FString&amp; Reason) override</i>                                                                | <i>Reason</i> - Allows users to specify why the component is stopping                   |
| <i>void Cleanup() override</i>                                                                                           |                                                                                         |
| <b>Getters</b>                                                                                                           |                                                                                         |
| <i>UFUNCTION(BlueprintCallable, Category = "Utility Component") const UABUtilityActionBase* GetCurrentAction() const</i> |                                                                                         |

|                                                                                                                                                                                                                              |                                                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>UFUNCTION(BlueprintCallable, Category = "Utility Component")</i><br><i>const UABUtilityActionBase*</i><br><i>GetLastSuccessfulAction() const</i>                                                                          |                                                                                                                                                                                               |
| <i>UFUNCTION(BlueprintCallable, Category = "Utility Component")</i><br><i>const EABActionSelectionMode&amp;</i><br><i>GetActionSelectionMode() const</i>                                                                     |                                                                                                                                                                                               |
| <i>const FTimerHandle&amp;</i><br><i>GetSuccessfulActionCooldownHandle() const</i>                                                                                                                                           |                                                                                                                                                                                               |
| <i>const</i><br><i>FAJBUtilityComponentAsyncData&amp;</i><br><i>GetAsyncData() const</i>                                                                                                                                     |                                                                                                                                                                                               |
| <i>FAJBUtilityComponentAsyncData&amp;</i><br><i>GetMutableAsyncData()</i>                                                                                                                                                    |                                                                                                                                                                                               |
| <b><i>Pause and Abort functions</i></b>                                                                                                                                                                                      |                                                                                                                                                                                               |
| <i>UFUNCTION(BlueprintCallable, Category = "Utility Component")</i><br><i>virtual void AbortCurrentAction(const EABUtilityAbortActionLogic&amp; AbortLogic)</i>                                                              | <i>AbortLogic</i> - Allow the user to specify what happens to the component when we abort the current Action - stopping the component, pausing the component, or choosing a new Action        |
| <i>UFUNCTION(BlueprintCallable, Category = "Utility Component")</i><br><i>virtual void</i><br><i>PauseLogicWithActionDecision(const FString&amp; Reason, const EABUtilityComponentPauseActionLogic &amp; ActionDecision)</i> | <i>Reason</i> - Allow the user to specify why we are pausing the logic<br><br><i>ActionDecision</i> - Allow the user to specify what happens to the current Action upon pausing the component |
| <b><i>Inlines</i></b>                                                                                                                                                                                                        |                                                                                                                                                                                               |
| <i>UFUNCTION(BlueprintCallable, Category = "Utility Component")</i><br><i>const bool</i><br><i>ShouldRestartLogicWhenResumingComponentLogic() const</i>                                                                      |                                                                                                                                                                                               |
| <i>UFUNCTION(BlueprintCallable, Category = "Utility Component")</i><br><i>const bool</i><br><i>IsComponentRunningAsynchronously() const</i>                                                                                  |                                                                                                                                                                                               |

|                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>UFUNCTION(BlueprintCallable,<br/>Category = "Utility Component")<br/>const bool<br/>IsUsingParallelRequirementChecking()<br/>const</i> |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>UFUNCTION(BlueprintCallable,<br/>Category = "Utility Component")<br/>const bool ShouldRetryOnFailure()<br/>const</i>                   |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>UFUNCTION(BlueprintCallable,<br/>Category = "Utility Component")<br/>const bool IsUsingConsiderationFactor()<br/>const</i>             |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Replication functions</b>                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>void<br/>GetLifetimeReplicatedProps(TArray&lt;FLifetimeProperty&gt;&amp; OutLifetimeProps)<br/>const override</i>                      | <i>OutLifetimeProps</i> - Used to track a property that is marked to be replicated for the lifetime of the actor channel. This doesn't mean the property will necessarily always be replicated, it just means: "check this property for replication for the life of the actor, and I don't want to think about it anymore". A secondary condition can also be used to skip replication based on the condition results |
| <i>UFUNCTION(BlueprintNativeEvent)<br/>void OnRep_CurrentAction()</i>                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>UFUNCTION(BlueprintNativeEvent)<br/>void OnRep_LastSuccessfulAction()</i>                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>UFUNCTION(BlueprintNativeEvent)<br/>void OnRep_CurrentRunStatus()</i>                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>virtual void<br/>OnRep_CurrentAction_Implementation(<br/>)</i>                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>virtual void<br/>OnRep_LastSuccessfulAction_Implementation()</i>                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>virtual void<br/>OnRep_CurrentRunStatus_Implementation()</i>                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                       |



| <b>Debugging</b>                                                                                                                                                                                          |                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>virtual void</i><br><i>DescribeSelfToGameplayDebugger(FGameplayDebuggerCategory* DebuggerCategory, const int32 ActionIndex) const</i>                                                                  | <i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCategory_UTILITY</i> currently being used in the Gameplay Debugger Menu<br><br><i>ActionIndex</i> - The current index of the available Actions to show the debugging for     |
| <b>Delegate binding functions</b>                                                                                                                                                                         |                                                                                                                                                                                                                                                |
| <i>UFUNCTION()</i><br><i>virtual void</i><br><i>OnActionCompleted(UABUtilityActionBase* Action, const EABUtilityActionRunStatus ActionStatus, const UABUtilityActionInstanceMemoryBase* ActionMemory)</i> | <i>Action</i> - The Action that triggered this instance of the callback<br><br><i>ActionStatus</i> - The final status of the Action for this component<br><br><i>ActionMemory</i> - The Instance Memory for the component for the given Action |
| <i>void</i><br><i>OnActionRequirementsChecked(UABUtilityActionBase* Action, const bool bRequirementsPassed)</i>                                                                                           | <i>Action</i> - The Action that triggered this instance of the callback<br><br><i>bRequirementsPassed</i> - Whether or not all of the Actions Requirements passed and therefore if the Action is possible right now                            |
| <b>Action Logic Functions</b>                                                                                                                                                                             |                                                                                                                                                                                                                                                |
| <i>bool PerformUtilityAction()</i>                                                                                                                                                                        |                                                                                                                                                                                                                                                |
| <i>bool RequestUtilityAction()</i>                                                                                                                                                                        |                                                                                                                                                                                                                                                |
| <i>void AsyncRequestUtilityAction()</i>                                                                                                                                                                   |                                                                                                                                                                                                                                                |
| <i>void</i><br><i>GetPossibleActions(TMap&lt;UABUtilityActionBase*, const float&gt;&amp; PossibleActionsToScores)</i>                                                                                     | <i>PossibleActionsToScores</i> - A map with the key as the Action and the value as the score for that Action                                                                                                                                   |
| <i>void</i><br><i>ParallelGetPossibleActions(TMap&lt;UABUtilityActionBase*, const float&gt;&amp; PossibleActionsToScores)</i>                                                                             | <i>PossibleActionsToScores</i> - A map with they key as the Action and the value as the score for that Action                                                                                                                                  |
| <i>void AsyncGetPossibleActions()</i>                                                                                                                                                                     |                                                                                                                                                                                                                                                |
| <i>void ParallelAsyncGetPossibleActions()</i>                                                                                                                                                             |                                                                                                                                                                                                                                                |

|                                                                                                                                                         |                                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <i>void</i><br><i>GetAvailableActionsFromSubsystem(TArray&lt;const UABUtilityActionBase*&gt;&amp; AvailableActions) const</i>                           | <i>AvailableActions</i> - An array of Actions available to this component instance |
| <i>void</i><br><i>GetAvailableMutableActionsFromSubsystem(TArray&lt;UABUtilityActionBase*&gt;&amp; AvailableActions)</i>                                | <i>AvailableActions</i> - An array of Actions available to this component instance |
| <i>const</i><br><i>UABUtilityActionInstanceMemoryBase*</i><br><i>GetActionInstanceMemoryFromSubsystem(const UABUtilityActionBase&amp; Action) const</i> | <i>Action</i> - The Action to get the Instance Memory for                          |
| <i>UABUtilityActionInstanceMemoryBase*</i><br><i>GetMutableActionInstanceMemoryFromSubsystem(const UABUtilityActionBase&amp; Action)</i>                | <i>Action</i> - The Action to get the Instance Memory for                          |
| <b>Retry logic</b>                                                                                                                                      |                                                                                    |
| <i>bool AttemptRetry()</i>                                                                                                                              |                                                                                    |

## Members:

|                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data assets</b>                                                                                                                   |
| <i>UPROPERTY(EditAnywhere, Category = "Utility")</i><br><i>TSoftObjectPtr&lt;UABUtilityDataAsset&gt; UtilityDataAssetClass</i>       |
| <b>Replicated properties</b>                                                                                                         |
| <i>UPROPERTY(ReplicatedUsing = OnRep_CurrentAction)</i><br><i>TObjectPtr&lt;UABUtilityActionBase&gt; CurrentAction</i>               |
| <i>UPROPERTY(ReplicatedUsing = OnRep_LastSuccessfulAction)</i><br><i>TObjectPtr&lt;UABUtilityActionBase&gt; LastSuccessfulAction</i> |
| <i>UPROPERTY(ReplicatedUsing = OnRep_CurrentRunStatus)</i><br><i>EABUtilityActionRunStatus CurrentRunStatus</i>                      |
| <b>Actions</b>                                                                                                                       |
| <i>UPROPERTY(EditAnywhere, Category = "Actions")</i><br><i>EABActionSelectionMode ActionSelectionMode</i>                            |
| <b>Inlines</b>                                                                                                                       |

|                                                                                                             |
|-------------------------------------------------------------------------------------------------------------|
| <i>UPROPERTY(EditAnywhere, Category = "Pausing")<br/>uint32 bRestartLogicWhenResumingComponentLogic : 1</i> |
| <i>UPROPERTY(EditAnywhere, Category = "Async")<br/>uint32 bRunComponentAsynchronously : 1</i>               |
| <i>UPROPERTY(EditAnywhere, Category = "Async")<br/>uint32 bUseParallelRequirementChecking : 1</i>           |
| <i>UPROPERTY(EditAnywhere, Category = "Utility")<br/>uint32 bRetryOnFailure : 1</i>                         |
| <i>UPROPERTY(EditAnywhere, Category = "Scoring")<br/>uint32 bUseConsiderationFactorScoreModifier</i>        |
| <b><i>Async functionality</i></b>                                                                           |
| <i>FAJBUtilityComponentAsyncData AsyncData</i>                                                              |
| <b><i>Timers</i></b>                                                                                        |
| <i>FTimerHandle SuccessfulActionCooldownHandle</i>                                                          |

**Other:**

|                                                       |
|-------------------------------------------------------|
| <b><i>Types</i></b>                                   |
| <i>struct FAJBUtilityComponentAsyncData</i>           |
| <i>enum class EABUtilityComponentPauseActionLogic</i> |

## UABUtilityActionBase

This is the base class for all Actions made both in this plugin and by others who may use this plugin. It contains Requirements for checking if it can run at all, Factors for calculating the score for the Action, and Tick functionality for any Actions that require running over multiple frames. Once the Action completes, it will trigger a delegate broadcast that the Utility Component should be tied to that will allow the component to find and run a new Action.

**Parent class:** *UObject*

**Functions:**

| Function                                                                                                              | Params                                                                              |
|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <i>UABUtilityActionBase()</i>                                                                                         |                                                                                     |
| <b>Initialization</b>                                                                                                 |                                                                                     |
| <i>virtual void Initialize(UWorld* InWorld)</i>                                                                       | <i>InWorld</i> - The instance of the World to cache and return in <i>GetWorld()</i> |
| <i>virtual void Deinitialize()</i>                                                                                    |                                                                                     |
| <b>Instance Memory</b>                                                                                                |                                                                                     |
| <i>virtual const<br/>TSubclassOf&lt;UABUtilityActionInstance<br/>MemoryBase&gt;<br/>GetInstanceMemoryType() const</i> |                                                                                     |
| <b>World operations</b>                                                                                               |                                                                                     |
| <i>UWorld* GetWorld()</i>                                                                                             |                                                                                     |
| <b>Action Requirement Checking</b>                                                                                    |                                                                                     |
| <i>const bool CheckAllRequirements(const<br/>UABUtilityComponent&amp; OwnerComp)<br/>const</i>                        | <i>OwnerComp</i> - The owning Utility Component                                     |
| <i>void AsyncCheckAllRequirements(const<br/>UABUtilityComponent&amp; OwnerComp)</i>                                   | <i>OwnerComp</i> - The owning Utility Component                                     |
| <b>Getters</b>                                                                                                        |                                                                                     |

|                                                                                                                                                                    |                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>const</i><br><i>TArray&lt;UABUtilityRequirementBase*&gt;&amp;</i><br><i>GetRequirements() const</i>                                                             |                                                                                                                                                                                          |
| <i>const TArray&lt;UABUtilityFactorBase*&gt;&amp;</i><br><i>GetFactors() const</i>                                                                                 |                                                                                                                                                                                          |
| <i>const float</i><br><i>GetOnSuccessCooldownValue(UABUtilityComponent&amp; OwnerComp) const</i>                                                                   | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                          |
| <i>const float GetUtilityWeightValue(const UABUtilityComponent&amp; OwnerComp) const</i>                                                                           | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                          |
| <b>Action scoring</b>                                                                                                                                              |                                                                                                                                                                                          |
| <i>const float GetActionScore(const UABUtilityComponent&amp; OwnerComp, const bool bUseConsiderationFactor) const</i>                                              | <i>OwnerComp</i> - The owning Utility Component<br><br><i>bUseConsiderationFactor</i> - Whether or not the scoring will be modified by the Consideration Factor                          |
| <b>Action logic</b>                                                                                                                                                |                                                                                                                                                                                          |
| <i>const EABUtilityActionRunStatus&amp;</i><br><i>PerformAction(UABUtilityComponent&amp; OwnerComp)</i>                                                            | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                          |
| <i>virtual void</i><br><i>OnActionCompleted(UABUtilityActionInstanceMemoryBase* InstanceMemory, const EABUtilityActionRunStatus ActionStatus)</i>                  | <i>InstanceMemory</i> - The Instance Memory for the component that completed the Action<br><br><i>ActionStatus</i> - The final status of the Action for a given component                |
| <b>Pause, Resume and Abort logic</b>                                                                                                                               |                                                                                                                                                                                          |
| <i>virtual void</i><br><i>PauseAction(UABUtilityComponent* OwnerComp, const FString&amp; Reason, UABUtilityActionInstanceMemoryBase* InstanceMemory = nullptr)</i> | <i>OwnerComp</i> - The owning Utility Component<br><br><i>Reason</i> - The reason the Action is being paused<br><br><i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i> |
| <i>virtual bool</i><br><i>ResumeAction(UABUtilityComponent* OwnerComp,</i>                                                                                         | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                          |

|                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>UABUtilityActionInstanceMemoryBase*</i><br><i>InstanceMemory = nullptr)</i>                                                                                                                                                                                                                                                 | <i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>                                                                                                                                                                      |
| <i>virtual void</i><br><i>AbortAction(UABUtilityComponent*</i><br><i>OwnerComp, const FString&amp; Reason,</i><br><i>UABUtilityActionInstanceMemoryBase*</i><br><i>InstanceMemory = nullptr)</i>                                                                                                                               | <i>OwnerComp</i> - The owning Utility Component<br><br><i>Reason</i> - The reason the Action is being aborted<br><br><i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>                                                 |
| <b>Tick logic</b>                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                           |
| <i>virtual EABUtilityActionRunStatus</i><br><i>TickAction(float DeltaTime,</i><br><i>UABUtilityComponent&amp; OwnerComp,</i><br><i>UABUtilityActionInstanceMemoryBase*</i><br><i>InstanceMemory)</i>                                                                                                                           | <i>DeltaTime</i> - The Tick time for this frame<br><br><i>OwnerComp</i> - The component that needs this Action to tick<br><br><i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>                                        |
| <b>Inlines</b>                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                           |
| <i>const bool</i><br><i>IsUsingParallelRequirementChecking()</i><br><i>const</i>                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                           |
| <i>const bool</i><br><i>IsConsideringFactorScoreZero() const</i>                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                           |
| <i>const bool ShouldTick() const</i>                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                           |
| <b>Validation</b>                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                           |
| <i>virtual const bool</i><br><i>ValidateAction(FString&amp;</i><br><i>InvalidationReason) const</i>                                                                                                                                                                                                                            | <i>InvalidationReason</i> - The string to be printed if anything about the Action is invalid                                                                                                                                              |
| <b>Debugging</b>                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                           |
| <i>virtual void</i><br><i>DescribeSelfToGameplayDebugger(con</i><br><i>st UABUtilityComponent&amp; OwnerComp,</i><br><i>const bool bUseConsiderationFactor,</i><br><i>const</i><br><i>UABUtilityActionInstanceMemoryBase*</i><br><i>InstanceMemory,</i><br><i>FGameplayDebuggerCategory*</i><br><i>DebuggerCategory) const</i> | <i>OwnerComp</i> - The owning Utility Component<br><br><i>bUseConsiderationFactor</i> - Whether or not the score will be modified by the consideration factor<br><br><i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i> |

|                                                                                                                                                           |                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                           | <i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCategory_UTILITY</i> currently being used in the Gameplay Debugger Menu                                             |
| <b>Internal Action logic</b>                                                                                                                              |                                                                                                                                                                                       |
| <i>virtual EABUtilityActionRunStatus DoAction(UABUtilityComponent&amp; OwnerComp, UABUtilityActionInstanceMemoryBase* InstanceMemory)</i>                 | <i>OwnerComp</i> - The owning Utility Component<br><br><i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>                                                           |
| <b>Internal Tick logic</b>                                                                                                                                |                                                                                                                                                                                       |
| <i>virtual EABUtilityActionRunStatus Tick(UABUtilityComponent&amp; OwnerComp, float DeltaSeconds, UABUtilityActionInstanceMemoryBase* InstanceMemory)</i> | <i>OwnerComp</i> - The owning Utility Component<br><br><i>DeltaSeconds</i> - The Tick time for this frame<br><br><i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i> |
| <b>Internal Action Requirement checking</b>                                                                                                               |                                                                                                                                                                                       |
| <i>const bool InternalCheckAllRequirements(const UABUtilityComponent&amp; OwnerComp) const</i>                                                            | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                       |
| <i>const bool InternalParallelCheckAllRequirements(const UABUtilityComponent&amp; OwnerComp) const</i>                                                    | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                       |
| <i>void InternalAsyncCheckAllRequirements(const UABUtilityComponent&amp; OwnerComp)</i>                                                                   | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                       |
| <i>void InternalParallelAsyncCheckAllRequirements(const UABUtilityComponent&amp; OwnerComp)</i>                                                           | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                       |
| <i>virtual void OnAsyncRequirementChecked(const UABUtilityRequirementBase* const Requirement, const bool bRequirementPassed)</i>                          | <i>Requirement</i> - The Requirement that triggered this instance of the callback<br><br><i>bRequirementPassed</i> - Whether or not the Requirement passed                            |

| <b>Internal Action scoring</b>                                                                                                                            |                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>void ModifyTotalScore(const float&amp; ModifyingScore, const float&amp; NumFactors, const int32&amp; CurrentIndex, float&amp; OutTotalScore) const</i> | <i>ModifyingScore</i> - The score to modify the total by<br><i>NumFactors</i> - The number of Factors held by the Action as a <i>float</i><br><i>CurrentIndex</i> - The index of the current Factor being calculated<br><i>OutTotalScore</i> - The score after the modifier has been applied |
| <b>Other</b>                                                                                                                                              |                                                                                                                                                                                                                                                                                              |
| <i>virtual const bool ShouldTickIfViewportsOnly() const</i>                                                                                               |                                                                                                                                                                                                                                                                                              |

#### Members:

| <b>Delegate broadcasts</b>                                                                                       |
|------------------------------------------------------------------------------------------------------------------|
| <i>FOnActionCompletedDelegate OnActionCompletedDelegate</i>                                                      |
| <i>FOnAsyncRequirementsChecked OnAsyncRequirementsCheckedDelegate</i>                                            |
| <b>Cached members</b>                                                                                            |
| <i>TObjectPtr&lt;UWorld&gt; CachedWorld</i>                                                                      |
| <b>Async functionality</b>                                                                                       |
| <i>FAJBUtilityActionAsyncData AsyncData</i>                                                                      |
| <b>Data providers</b>                                                                                            |
| <i>UPROPERTY(EditAnywhere, meta = (ClampMin = 0.0, UIMin = 0.0)) FAIDataProviderFloatValue UtilityWeight</i>     |
| <i>UPROPERTY(EditAnywhere, meta = (ClampMin = 0.0, UIMin = 0.0)) FAIDataProviderFloatValue OnSuccessCooldown</i> |
| <b>Scoring</b>                                                                                                   |
| <i>UPROPERTY(EditAnywhere) EABFactorScoreTotallingMethod ScoreAggregationMethod</i>                              |
| <b>Requirements and Factors</b>                                                                                  |



|                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------|
| <i>UPROPERTY(EditAnywhere, Instanced)</i><br><i>TArray&lt;TObjectPtr&lt;UABUtilityRequirementBase&gt;&gt; Requirements</i> |
| <i>UPROPERTY(EditAnywhere, Instanced)</i><br><i>TArray&lt;TObjectPtr&lt;UABUtilityFactorBase&gt;&gt; Factors</i>           |
| <b><i>Inlines</i></b>                                                                                                      |
| <i>UPROPERTY(EditAnywhere)</i><br><i>uint32 bUseParallelRequirementChecking : 1</i>                                        |
| <i>UPROPERTY(EditAnywhere)</i><br><i>uint32 bConsiderFactorScoreZero : 1</i>                                               |
| <i>UPROPERTY(EditAnywhere, Category = "Utility AI Action Base")</i><br><i>uint32 bShouldTick : 1</i>                       |
| <b><i>Other</i></b>                                                                                                        |
| <i>UPROPERTY(EditAnywhere)</i><br><i>bool bShouldTickIfViewportsOnly</i>                                                   |

**Other:**

|                                                 |
|-------------------------------------------------|
| <b><i>Types</i></b>                             |
| <i>struct FAJBUtilityActionAsyncData</i>        |
| <i>enum class EABFactorScoreTotallingMethod</i> |

## UABUtilityRequirementBase

Requirements are used to check if an Action is able to run at that moment. When the Utility Component tries to run a new Action, it checks the Requirements of that Action first. They must all pass for the Action to be possible when checked. The only case where not every Requirement needs to pass is if the user adds a Multi-Requirement to the Action that groups with an OR operator instead of an AND operator.

**Parent class:** *UObject*

**Functions:**

| Function                                                                                                                                                        | Params                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Requirement checking</b>                                                                                                                                     |                                                                                                                                                                                                  |
| <i>virtual const bool<br/>CheckRequirement(const<br/>UABUtilityComponent&amp; OwnerComp)<br/>const</i>                                                          | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                                  |
| <i>void AsyncCheckRequirement(const<br/>UABUtilityComponent&amp; OwnerComp)<br/>const</i>                                                                       | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                                  |
| <b>World functions</b>                                                                                                                                          |                                                                                                                                                                                                  |
| <i>UWorld* GetWorld() const override</i>                                                                                                                        |                                                                                                                                                                                                  |
| <b>Validation</b>                                                                                                                                               |                                                                                                                                                                                                  |
| <i>virtual const bool<br/>ValidateRequirement(FString&amp;<br/>InvalidationReason) const</i>                                                                    | <i>InvalidationReason</i> - The string to be printed if anything about the Requirement is invalid                                                                                                |
| <b>Debugging</b>                                                                                                                                                |                                                                                                                                                                                                  |
| <i>virtual void<br/>DescribeSelfToGameplayDebugger(const<br/>UABUtilityComponent&amp; OwnerComp,<br/>FGameplayDebuggerCategory*<br/>DebuggerCategory) const</i> | <i>OwnerComp</i> - The owning Utility Component<br><br><i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCategory_UTILITY</i> currently being used in the Gameplay Debugger Menu |

**Members:**

| <i><b>Delegate broadcasts</b></i>                                   |
|---------------------------------------------------------------------|
| <i>FOnAsyncRequirementChecked OnAsyncRequirementCheckedDelegate</i> |

## UABUtilityFactorBase

Factors are used to calculate the score for an Action when the Utility Component has already checked its Requirements and is considering it as a possibility. Each Factor has its own score that is evaluated when the Action is having its score calculated, then all of an Actions scores are totalled using a user-set method. If a Factors score returns 0, it will usually be ignored unless the user has enabled

*bConsiderFactorScoreZero* on the owning Action. The Factor has an optional extra bit of functionality called the “Consideration Factor”, which is inspired by and

described well in this video: [GDC Vault - Building a Better Centaur: AI at Massive Scale](#)

**Parent class:** *UObject*

**Functions:**

| Function                                                                                                                                           | Params                                                                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Scoring</b>                                                                                                                                     |                                                                                                                                                                                                                                                                                  |
| <i>const float GetFactorScore(const UABUtilityComponent&amp; OwnerComp, const float NumberOfFactors, const bool bUseConsiderationFactor) const</i> | <i>OwnerComp</i> - The owning Utility Component<br><br><i>NumberOfFactors</i> - The number of Factors owned by the owning Action, used by the Consideration Factor<br><br><i>bUseConsiderationFactor</i> - Whether or not the score will be modified by the Consideration Factor |
| <b>Inlines</b>                                                                                                                                     |                                                                                                                                                                                                                                                                                  |
| <i>const bool IsInversed() const</i>                                                                                                               |                                                                                                                                                                                                                                                                                  |
| <b>World functions</b>                                                                                                                             |                                                                                                                                                                                                                                                                                  |
| <i>UWorld* GetWorld() const override</i>                                                                                                           |                                                                                                                                                                                                                                                                                  |
| <b>Validation</b>                                                                                                                                  |                                                                                                                                                                                                                                                                                  |
| <i>virtual const bool</i>                                                                                                                          | <i>InvalidationReason</i> - The string to be                                                                                                                                                                                                                                     |

|                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>ValidateFactor(FString&amp; InvalidationReasons) const</i>                                                                                                                                                    | printed if anything about the Factor is invalid                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Debugging</b>                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <i>virtual void DescribeSelfToGameplayDebugger(const UABUtilityComponent&amp; OwnerComp, const float NumberOfFactors, const bool bUseConsiderationFactor, FGameplayDebuggerCategory* DebuggerCategory) const</i> | <p><i>OwnerComp</i> - The owning Utility Component</p> <p><i>NumberOfFactors</i> - The number of Factors owned by the owning Action, is used by the Consideration Factor</p> <p><i>bUseConsiderationFactor</i> - Whether or not the score will be modified by the Consideration Factor</p> <p><i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCategory_UTILITY</i> currently being used in the Gameplay Debugger</p> |
| <b>Internal scoring</b>                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <i>virtual const float CalculateFactorScore(const UABUtilityComponent&amp; OwnerComp) const</i>                                                                                                                  | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                                                                                                                                                                                                                                                                        |
| <i>void AddConsiderationFactor(const float NumberOfFactors, float&amp; OutFloat) const</i>                                                                                                                       | <p><i>NumberOfFactors</i> - The number of Factors owned by the owning Action</p> <p><i>OutFloat</i> - The Factor score to be used in the calculation and set at the end of the logic</p>                                                                                                                                                                                                                                               |

## Members:

|                                                                                         |
|-----------------------------------------------------------------------------------------|
| <b>Inlines</b>                                                                          |
| <i>UPROPERTY(EditAnywhere, Category = "Scoring")</i><br><i>uint32 bInverseScore : 1</i> |

## UABUtilityActionBlueprintBase

This Action is the base for any Actions that users wish to make in Blueprints instead of C++. It overrides the relevant functions from the Action base, and adds Blueprint functions that allow users to validate, debug, run and Tick an Action.

**Parent class:** *UABUtilityActionBase*

### Functions:

| Function                                                                                                                                                                                                                                       | Params                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Validation</b>                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                |
| <i>const bool ValidateAction(FString&amp; InInvalidationReason) const override</i>                                                                                                                                                             | <i>InInvalidationReason</i> - The string to be printed if anything about the Action is invalid                                                                                                                                                                                                 |
| <i>UFUNCTION(BlueprintNativeEvent)<br/>const bool ValidateBlueprintAction(const FString&amp; InInvalidationReason, FString&amp; OutInvalidationReason) const</i>                                                                               | <i>InInvalidationReason</i> - The string that already exists to be printed<br><br><i>OutInvalidationReason</i> - The string to be printed if anything about the Action is invalid, should contain the string passed into <i>InInvalidationReason</i>                                           |
| <i>virtual const bool ValidateBlueprintAction_Implementation(const FString&amp; InInvalidationReason, FString&amp; OutInvalidationReason) const</i>                                                                                            | <i>InInvalidationReason</i> - The string that already exists to be printed<br><br><i>OutInvalidationReason</i> - The string to be printed if anything about the Action is invalid, should contain the string passed into <i>InInvalidationReason</i>                                           |
| <b>Debugging</b>                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                |
| <i>void DescribeSelfToGameplayDebugger(const UABUtilityComponent&amp; OwnerComp, const bool bUseConsiderationFactor, const UABUtilityActionInstanceMemoryBase* InstanceMemory, FGameplayDebuggerCategory* DebuggerCategory) const override</i> | <i>OwnerComp</i> - The owning Utility Component<br><br><i>bUseConsiderationFactor</i> - Whether or not the score will be modified by the Consideration Factor<br><br><i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i><br><br><i>DebuggerCategory</i> - The instance of the |

|                                                                                                                                                                                |                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                | <i>FGameplayDebuggerCategory_Utility</i><br>currently being used in the Gameplay Debugger Menu                                                                                                   |
| <i>UFUNCTION(BlueprintNativeEvent)</i><br><i>const FString</i><br><i>DescribeBlueprintSelfToGameplayDebugger(const UABUtilityComponent* const OwnerComp) const</i>             | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                                  |
| <i>virtual const FString</i><br><i>DescribeBlueprintSelfToGameplayDebugger_Implementation(const UABUtilityComponent* const OwnerComp) const</i>                                | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                                  |
| <b>Internal Action logic</b>                                                                                                                                                   |                                                                                                                                                                                                  |
| <i>EABUtilityActionRunStatus</i><br><i>DoAction(UABUtilityComponent&amp; OwnerComp,</i><br><i>UABUtilityActionInstanceMemoryBase* InstanceMemory) override</i>                 | <i>OwnerComp</i> - The owning Utility Component<br><br><i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>                                                                      |
| <i>UFUNCTION(BlueprintNativeEvent)</i><br><i>EABUtilityActionRunStatus</i><br><i>DoBlueprintAction(UABUtilityComponent* OwnerComp)</i>                                         | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                                  |
| <i>virtual EABUtilityActionRunStatus</i><br><i>DoBlueprintAction_Implementation(UABUtilityComponent* OwnerComp)</i>                                                            | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                                  |
| <b>Internal Tick logic</b>                                                                                                                                                     |                                                                                                                                                                                                  |
| <i>EABUtilityActionRunStatus</i><br><i>Tick(UABUtilityComponent&amp; OwnerComp, float DeltaSeconds,</i><br><i>UABUtilityActionInstanceMemoryBase* InstanceMemory) override</i> | <i>OwnerComp</i> - The owning Utility Component<br><br><i>DeltaSeconds</i> - The Tick time in seconds for this frame<br><br><i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i> |
| <i>UFUNCTION(BlueprintNativeEvent)</i><br><i>EABUtilityActionRunStatus</i><br><i>TickBlueprintAction(UABUtilityComponent* OwnerComp, float DeltaSeconds)</i>                   | <i>OwnerComp</i> - The owning Utility Component<br><br><i>DeltaSeconds</i> - The Tick time in seconds for this frame                                                                             |
| <i>virtual EABUtilityActionRunStatus</i>                                                                                                                                       | <i>OwnerComp</i> - The owning Utility                                                                                                                                                            |

|                                                                                                        |                                                                                   |
|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <i>TickBlueprintAction_Implementation(UA<br/>BUtilityComponent* OwnerComp, float<br/>DeltaSeconds)</i> | Component<br><br><i>DeltaSeconds</i> - The Tick time in<br>seconds for this frame |
|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|



## UABUtilityRequirementBlueprintBase

This Requirement is the base for any Requirements that users wish to make in Blueprint instead of C++. It overrides the relevant functions from the Requirement base, and adds Blueprint functions that allow users to validate, debug and check the Requirement.

**Parent class:** *UABUtilityRequirementBase*

### Functions:

| Function                                                                                                                                                 | Params                                                                                                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Requirement checking</b>                                                                                                                              |                                                                                                                                                                                                                                                       |
| <i>const bool CheckRequirement(const UABUtilityComponent&amp; OwnerComp) const override</i>                                                              | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                                                                                       |
| <b>Validation</b>                                                                                                                                        |                                                                                                                                                                                                                                                       |
| <i>const bool ValidateRequirement(FString&amp; InInvalidationReason) const override</i>                                                                  | <i>InInvalidationReason</i> - The string to be printed if anything about the Requirement is invalid                                                                                                                                                   |
| <i>const bool ValidateBlueprintRequirement(const FString&amp; InInvalidationReason, FString&amp; OutInvalidationReason) const</i>                        | <i>InInvalidationReason</i> - The string that already exists to be printed<br><i>OutInvalidationReason</i> - The string to be printed if anything about the Requirement is invalid, should contain the string passed into <i>InInvalidationReason</i> |
| <i>virtual const bool ValidateBlueprintRequirement_Implementation(const FString&amp; InInvalidationReason, FString&amp; OutInvalidationReason) const</i> | <i>InInvalidationReason</i> - The string that already exists to be printed<br><i>OutInvalidationReason</i> - The string to be printed if anything about the Requirement is invalid, should contain the string passed into <i>InInvalidationReason</i> |
| <b>Debugging</b>                                                                                                                                         |                                                                                                                                                                                                                                                       |
| <i>void DescribeSelfToGameplayDebugger(con</i>                                                                                                           | <i>OwnerComp</i> - The owning Utility Component                                                                                                                                                                                                       |

|                                                                                                                                                |                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <i>st UABUtilityComponent&amp; OwnerComp, FGameplayDebuggerCategory* DebuggerCategory) const override</i>                                      | <i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCategory_UTILITY</i> currently being used in the Gameplay Debugger Menu |
| <i>UFUNCTION(BlueprintNativeEvent) const FString DescribeBlueprintSelfToGameplayDebugger(const UABUtilityComponent* const OwnerComp) const</i> | <i>OwnerComp</i> - The owning Utility Component                                                                                           |
| <i>virtual const FString DescribeBlueprintSelfToGameplayDebugger_Implementation(const UABUtilityComponent* const OwnerComp) const</i>          | <i>OwnerComp</i> - The owning Utility Component                                                                                           |
| <b><i>Internal requirement checking</i></b>                                                                                                    |                                                                                                                                           |
| <i>UFUNCTION(BlueprintNativeEvent) const bool CheckBlueprintRequirement(const UABUtilityComponent* const OwnerComp) const</i>                  | <i>OwnerComp</i> - The owning Utility Component                                                                                           |
| <i>virtual const bool CheckBlueprintRequirement_Implementation(const UABUtilityComponent* const OwnerComp) const</i>                           | <i>OwnerComp</i> - The owning Utility Component                                                                                           |

## UABUtilityFactorBlueprintBase

This Factor is the base for any Factors that users wish to make in Blueprint instead of C++. It overrides the relevant functions from the Factor base, and adds Blueprint functions that allow users to validate, debug and calculate the Factor score.

**Parent class:** *UABUtilityFactorBase*

### Functions:

| Function                                                                                                                                                                                                              | Params                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Validation</b>                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                     |
| <i>const bool ValidateFactor(FString&amp; InInvalidationReasons) const override</i>                                                                                                                                   | <i>InInvalidationReason</i> - The string to be printed if anything about the Factor is invalid                                                                                                                                                                                                      |
| <i>UFUNCTION(BlueprintNativeEvent)<br/>const bool<br/>ValidateBlueprintFactor(const FString&amp; InInvalidationReason, FString&amp; OutInvalidationReason) const</i>                                                  | <i>InInvalidationReason</i> - The string that already exists to be printed<br><br><i>OutInvalidationReason</i> - The string to be printed if anything about the Factor is invalid, should contain the string passed into <i>InInvalidationReason</i>                                                |
| <i>virtual const bool<br/>ValidateBlueprintFactor_Implementation(const FString&amp; InInvalidationReason, FString&amp; OutInvalidationReason) const</i>                                                               | <i>InInvalidationReason</i> - The string that already exists to be printed<br><br><i>OutInvalidationReason</i> - The string to be printed if anything about the Factor is invalid, should contain the string passed into <i>InInvalidationReason</i>                                                |
| <b>Debugging</b>                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                     |
| <i>void<br/>DescribeSelfToGameplayDebugger(const UABUtilityComponent&amp; OwnerComp, const float NumberOfFactors, const bool bUseConsiderationFactor, FGameplayDebuggerCategory* DebuggerCategory) const override</i> | <i>OwnerComp</i> - The owning Utility Component<br><br><i>NumberOfFactors</i> - The number of Factors owned by the owning Action<br><br><i>bUseConsiderationFactor</i> - Whether or not the score will be modified by the Consideration Factor<br><br><i>DebuggerCategory</i> - The instance of the |

|                                                                                                                                                                                                 |                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                 | <i>FGameplayDebuggerCategory_Utility</i><br>currently being used in the Gameplay Debugger Menu                                   |
| <i>UFUNCTION(BlueprintNativeEvent)</i><br><i>const FString</i><br><i>DescribeBlueprintSelfToGameplayDebugger(const UABUtilityComponent* const OwnerComp, const float NumberOfFactors) const</i> | <i>OwnerComp</i> - The owning Utility Component<br><br><i>NumberOfFactors</i> - The number of Factors owned by the owning Action |
| <i>virtual const FString</i><br><i>DescribeBlueprintSelfToGameplayDebugger_Implementation(const UABUtilityComponent* const OwnerComp, const float NumberOfFactors) const</i>                    | <i>OwnerComp</i> - The owning Utility Component<br><br><i>NumberOfFactors</i> - The number of Factors owned by the owning Action |
| <b>Internal scoring</b>                                                                                                                                                                         |                                                                                                                                  |
| <i>const float CalculateFactorScore(const UABUtilityComponent&amp; OwnerComp) const override</i>                                                                                                | <i>OwnerComp</i> - The owning Utility Component                                                                                  |
| <i>UFUNCTION(BlueprintNativeEvent)</i><br><i>const float</i><br><i>CalculateBlueprintFactorScore(const UABUtilityComponent* const OwnerComp) const</i>                                          | <i>OwnerComp</i> - The owning Utility Component                                                                                  |
| <i>virtual const float</i><br><i>CalculateBlueprintFactorScore_Implementation(const UABUtilityComponent* const OwnerComp) const</i>                                                             | <i>OwnerComp</i> - The owning Utility Component                                                                                  |

## UABUtilityDataAsset

This is the data asset used to define the Actions available to an AI. It allows users to define a list of Actions and fill out the data for those Actions, as well as define lists of Requirements and Factors for those Actions and fill out the data for those as well.

**Parent class:** *UDataAsset*

### Functions:

| Function                                                                                      | Params |
|-----------------------------------------------------------------------------------------------|--------|
| <b>Actions</b>                                                                                |        |
| <i>const<br/>TArray&lt;TObjectPtr&lt;UABUtilityActionBase&gt;&gt;&amp; GetActions() const</i> |        |
| <b>Scoring</b>                                                                                |        |
| <i>const bool IsUsingConsiderationFactor()<br/>const</i>                                      |        |

### Members:

|                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------|
| <b>Actions</b>                                                                                                    |
| <i>UPROPERTY(EditAnywhere, Instanced)<br/>TArray&lt;TObjectPtr&lt;UABUtilityActionBase&gt;&gt; UtilityActions</i> |

## UABUtilitySettings

This is a Developer Settings class for the Utility plugin, containing assets that cannot be exposed on the subsystem and other global variables.

**Parent class:** *UDeveloperSettings*

### Functions:

| Function                                                                                                         | Params |
|------------------------------------------------------------------------------------------------------------------|--------|
| <i>const TSet&lt;const TSoftObjectPtr&lt;const UABUtilityDataAsset&gt;&gt;&amp; GetUtilityDataAssets() const</i> |        |
| <i>const bool ShouldPrintCallstackInLogs() const</i>                                                             |        |

### Members:

|                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data Assets</b>                                                                                                                                       |
| <i>UPROPERTY(EditAnywhere, Config, Category = "Data Assets")<br/>TSet&lt;const TSoftObjectPtr&lt;const UABUtilityDataAsset&gt;&gt; UtilityDataAssets</i> |
| <b>Debugging</b>                                                                                                                                         |
| <i>UPROPERTY(EditAnywhere, Config, Category = "Debugging")<br/>bool bPrintCallstackInLogs = true</i>                                                     |

## UABUtilitySubsystem

This subsystem holds all instances of Actions and Instance Memory, and allows Utility Components to subscribe to Actions by supplying an unloaded Data Asset that the subsystem gets the Actions from, creates Instance Memory for, and stores locally.

**Parent class:** *UTickableWorldSubsystem*

### Functions:

| Function                                                                                                                                                   | Params                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>void Initialize(FSubsystemCollectionBase&amp; Collection) override</i>                                                                                  | <i>Collection</i> - The collection of subsystems                                                                                                                         |
| <i>void Deinitialize() override</i>                                                                                                                        |                                                                                                                                                                          |
| <i>void Tick(float DeltaTime) override</i>                                                                                                                 | <i>DeltaTime</i> - The Tick time for this frame                                                                                                                          |
| <i>TStatId GetStatId() const override</i>                                                                                                                  |                                                                                                                                                                          |
| <i>const bool SubscribeToDataAsset(UABUtilityComponent&amp; SubscribingComponent, const TSoftObjectPtr&lt;UABUtilityDataAsset&gt;&amp; DataAssetClass)</i> | <i>SubscribingComponent</i> - The Utility Component subscribing to Actions<br><br><i>DataAssetClass</i> - The unloaded Data Asset containing the Actions to subscribe to |
| <i>const bool Unsubscribe(const UABUtilityComponent&amp; UnsubscribingComponent)</i>                                                                       | <i>UnsubscribingComponent</i> - The Utility Component unsubscribing from Actions                                                                                         |
| <i>const TArray&lt;const UABUtilityActionBase*&gt; GetActionsAvailableToComponent(const UABUtilityComponent&amp; Component) const</i>                      | <i>Component</i> - The Utility Component to get the available Actions for                                                                                                |
| <i>TArray&lt;UABUtilityActionBase*&gt; GetMutableActionsAvailableToComponent(const UABUtilityComponent&amp; Component) const</i>                           | <i>Component</i> - The Utility Component to get the available Actions for                                                                                                |
| <i>const UABUtilityActionInstanceMemoryBase*</i>                                                                                                           | <i>Action</i> - The Action to find the Instance Memory Type for                                                                                                          |

|                                                                                                                                                                                       |                                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>GetActionInstanceMemoryForComponent(const UABUtilityActionBase&amp; Action, const UABUtilityComponent&amp; Component) const</i>                                                    | <i>Component</i> - The Component to find Instance Memory for                                                                                                    |
| <i>UABUtilityActionInstanceMemoryBase* GetMutableActionInstanceMemoryForComponent(const UABUtilityActionBase&amp; Action, const UABUtilityComponent&amp; Component)</i>               | <i>Action</i> - The Action to find the Instance Memory Type for<br><br><i>Component</i> - The Component to find the Instance Memory for                         |
| <i>const UABUtilityComponent* GetComponentForActionInstanceMemory(const UABUtilityActionBase&amp; Action, const UABUtilityActionInstanceMemoryBase&amp; InstanceMemory) const</i>     | <i>Action</i> - The Action to find a subscribed Component for<br><br><i>InstanceMemory</i> - The Instance Memory to use to find a specific subscribed Component |
| <i>UABUtilityComponent* GetMutableComponentForActionInstanceMemory(const UABUtilityActionBase&amp; Action, const UABUtilityActionInstanceMemoryBase&amp; InstanceMemory) const</i>    | <i>Action</i> - The Action to find a subscribed Component for<br><br><i>InstanceMemory</i> - The Instance Memory to use to find a specific subscribed Component |
| <i>bool DoesSupportWorldType(const EWorldType::Type WorldType) const override</i>                                                                                                     | <i>WorldType</i> - The type of world being initialized                                                                                                          |
| <i>UABUtilityActionInstanceMemoryBase* InternalGetMutableActionInstanceMemoryForComponent(const UABUtilityActionBase&amp; Action, const UABUtilityComponent&amp; Component) const</i> | <i>Action</i> - The Action to find the Instance Memory Type for<br><br><i>Component</i> - The Component to find the Instance Memory for                         |

## Members:

| <b>Actions</b>                                                                                 |
|------------------------------------------------------------------------------------------------|
| <i>TMap&lt;UABUtilityActionBase*, FInstanceMemoryWrapper&gt; ActionComponentInstanceMemory</i> |



## FGameplayDebuggerCategory\_Utility

The custom Gameplay Debugger Category used for debugging the Utility

Component, Actions, Requirements and Factors.

**Parent class:** *FGameplayDebuggerCategory*

### Functions:

| Function                                                                         | Params                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Debugging</b>                                                                 |                                                                                                                                                                                                                                         |
| <i>FGameplayDebuggerCategory_Utility()</i>                                       |                                                                                                                                                                                                                                         |
| <i>void CollectData(APlayerController* OwnerPC, AActor* DebugActor) override</i> | <i>OwnerPC</i> - The player controller for the user that enabled the Gameplay Debugger Menu<br><br><i>DebugActor</i> - The actor in the world that is being debugged and having its information displayed in the Gameplay Debugger Menu |
| <i>static TSharedRef&lt;FGameplayDebuggerCategory&gt; MakeInstance()</i>         |                                                                                                                                                                                                                                         |
| <i>void OnChangeActionIndex()</i>                                                |                                                                                                                                                                                                                                         |

### Members:

|                          |
|--------------------------|
| <b>Actions</b>           |
| <i>int32 ActionIndex</i> |

## UABUtilityDataAssetValidator

The editor validation class that validates the Utility Data Asset that has just been saved.

**Parent class:** *UEditorValidatorBase*

**Functions:**

| Function                                                                                                                                                                     | Params                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Validation</b>                                                                                                                                                            |                                                                                                                                                                                                                                                                  |
| <i>bool</i><br><i>CanValidateAsset_Implementation(const FAssetData&amp; InAssetData, UObject* InObject, FDataValidationContext&amp; InContext) const override</i>            | <i>InAssetData</i> - Holds important information about the asset found by the Asset Registry<br><br><i>InObject</i> - The asset that is being validated<br><br><i>Context</i> - The interface between <i>UObject::IsDataValid</i> and the data validation system |
| <i>EDataValidationResult</i><br><i>ValidateLoadedAsset_Implementation(const FAssetData&amp; InAssetData, UObject* InAsset, FDataValidationContext&amp; Context) override</i> | <i>InAssetData</i> - Holds important information about the asset found by the Asset Registry<br><br><i>InAsset</i> - The asset that is being validated<br><br><i>Context</i> - The interface between <i>UObject::IsDataValid</i> and the data validation system  |

## ABUtilityTypes.h

This is a helper file containing types that exist across multiple classes.

| Type                                                                       |
|----------------------------------------------------------------------------|
| <i>UENUM(BlueprintType)</i><br><i>enum class EABUtilityActionRunStatus</i> |
| <i>enum class EABUtilityAbortActionLogic</i>                               |
| <i>UENUM(BlueprintType)</i><br><i>enum class EABActionSelectionMode</i>    |

## ABUtilityHelpers.h

This is a helper file containing namespaces and classes that are used by multiple classes and/or structs.

### Namespaces:

| <b><i>ABUtility::Helpers::Requirements</i></b>                                                                                                               |                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>                                                                                                                                              | <b>Params</b>                                                                                                                                                       |
| <i>bool CompareValues(const float Left, const float Right, const EGenericAICheck&amp; ComparisonType)</i>                                                    | <i>Left</i> - The first value to be compared<br><br><i>Right</i> - The second value to be compared<br><br><i>ComparisonType</i> - How the values should be compared |
| <i>const UABUtilitySubsystem* GetUtilitySubsystem(const UObject* WorldContextObject)</i>                                                                     | <i>WorldContextObject</i> - An object with <i>GetWorld()</i> implemented                                                                                            |
| <i>const UABUtilitySubsystem&amp; GetUtilitySubsystemChecked(const UObject* WorldContextObject)</i>                                                          | <i>WorldContextObject</i> - An object with <i>GetWorld()</i> implemented                                                                                            |
| <i>UABUtilitySubsystem* GetMutableUtilitySubsystem(const UObject* WorldContextObject)</i>                                                                    | <i>WorldContextObject</i> - An object with <i>GetWorld()</i> implemented                                                                                            |
| <i>UABUtilitySubsystem&amp; GetMutableUtilitySubsystemChecked(const UObject* WorldContextObject)</i>                                                         | <i>WorldContextObject</i> - An object with <i>GetWorld()</i> implemented                                                                                            |
| <i>template &lt;typename T&gt; const T* GetActionInstanceMemoryOfType(const UABUtilityActionBase&amp; Action, const UABUtilityComponent&amp; Component)</i>  | <i>Action</i> - The Action to get the Instance Memory Type for<br><br><i>Component</i> - The Utility Component to get the Instance Memory for                       |
| <i>template &lt;typename T&gt; T* GetMutableActionInstanceMemoryOfType(const UABUtilityActionBase&amp; Action, const UABUtilityComponent&amp; Component)</i> | <i>Action</i> - The Action to get the Instance Memory Type for<br><br><i>Component</i> - The Utility Component to get the Instance Memory for                       |
| <i>const FString</i>                                                                                                                                         | <i>IgnoreCount</i> - The amount of currently                                                                                                                        |

|                                                                                                                     |                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <i>GetDebugCallstackString(int32 IgnoreCount)</i>                                                                   | stacked functions to ignore in the callstack                                                                                |
| <i>void UtilityVLOG(const ELogVerbosity::Type Verbosity, const FString&amp; Message, const UObject* LogOwner)</i>   | <i>Verbosity</i> - The level of log<br><i>Message</i> - The log to print<br><i>LogOwner</i> - The owning object for the log |
| <i>EABUtilityActionRunStatus StateTreeRunStatusToUtilityActionRunStatus(EStateTreeRunStatus StateTreeRunStatus)</i> | <i>StateTreeRunStatus</i> - The <i>EStateTreeRunStatus</i> to convert to an <i>EABUtilityActionRunStatus</i>                |
| <i>const FVector::FReal CalculateAngleDifferenceDot(const FVector&amp; A, const FVector&amp; B)</i>                 | <i>A</i> - Initial vector<br><i>B</i> - Secondary vector                                                                    |

## Examples

### Component Setup

The first thing to set up is an AI Controller. Create a custom class that inherits from *AAIController*, expose a pointer to a *UABUtilityComponent* and in the constructor, create the component and set it as the *BrainComponent*.

#### **.h file**

```
UCLASS()
class AJIMBOBUTILITYAI_API ATestUtilAIController : public AAIController
{
    GENERATED_BODY()

public:
    ATestUtilAIController(const FObjectInitializer& ObjectInitializer);

private:
    UPROPERTY(EditAnywhere)
    TObjectPtr<UABUtilityComponent> UtilityComponent = nullptr;
};
```

#### **.cpp file**

```
ATestUtilAIController::ATestUtilAIController(const FObjectInitializer&
ObjectInitializer)
{
    bStartAILogicOnPossess = true;
    bStopAILogicOnUnposses = true;

    UtilityComponent =
CreateDefaultSubobject<UABUtilityComponent>(TEXT("UtilityComponent"));
    BrainComponent = UtilityComponent;
}
```

Create a Blueprint of this custom AI Controller and, in the *UtilityComponent*, find the *UtilityDataAsset*. This is where the data asset will go once created.

Create a new data asset of type *UABUtilityDataAsset*. Open and save it, and set up the component so that *UtilityDataAsset* is set to this new asset. Now the asset can be built up.

In the component, find the *ActionSelectionMode* dropdown and set this to whichever mode most suits this AI. Now the asset can be filled with Actions.

## Code Action

To create an Action in code, go to the code files and create a new class that inherits from *UABUtilityActionBase*. Name it something that reflects what the Action should make the AI do.

Override *DoAction()*, this is the function where the main logic for the Action will exist.

If the Action requires Ticking, also override *Tick()* as the ticked logic will exist here.

In the *.cpp* file for this class, build up these two functions so that the Action does what is desired. If the Action should have other custom logic, functions such as *PauseAction()*, *ResumeAction()*, and *AbortAction()* can also be overridden to add custom logic to these processes.

If exposed members are being defined in the class, make sure to override *ValidateAction()* to ensure that the member is always valid (this will do editor-time validation when the asset is saved). Also override

*DescribeSelfToGameplayDebugger()* to ensure the value of the member can be shown in the Gameplay Debugger Menu for easier debugging.

Internal members and functions can be added freely to keep the Action code tidy and customisable to users.

## Blueprint Action

Another option for creating an Action is to create one in Blueprint. To do this, create a new Blueprint class of type *UABUtilityActionBlueprintBase* and name it something that reflects the purpose of the Action.

In this Action, override the function *DoBlueprintAction()* to define main Action logic, *TickBlueprintAction()* to define any logic that needs to be ticked, and if validation or debugging is desired then the functions *ValidateBlueprintAction()* and *DescribeBlueprintSelfToGameplayDebugger()* can be overridden in the Blueprint to supply these.

## Action Setup in Data Asset

In the data asset, add the new Action into the list of Actions. Any custom members will appear under the name of the Action. Base Action settings can be found under the *ABUtility Action Base* section. The most important of these is the *Utility Weight*, as if this is 0 the Action will never run. Set this to a value suitable compared to all Utility Weights for all Actions in this data asset.

The next thing to set up is the Requirements and Factors for the Action.

## Code Requirement

To create a Requirement in code, go to the code files and create a new class that inherits from *UABUtilityRequirementBase*. Name it something that reflects what the Requirement is checking.

Override the *CheckRequirement()* function and set it up so that it checks the correct thing. Custom functions and members can be added to ensure code cleanliness and customisability, and members can be validated and debugged in the



*ValidateRequirement()* and *DescribeSelfToGameplayDebugger()* function overrides respectively.

## Blueprint Requirement

Requirements can also be made in Blueprints. Create a new Blueprint class of type *UABUtilityRequirementBlueprintBase* and, in the new Blueprint, override *CheckBlueprintRequirement()* and set up the logic to check for what is required. Custom members can be validated and debugged in the overridable functions *ValidateBlueprintRequirement()* and *DescribeBlueprintSelfToGameplayDebugger()*.

## Multi-Requirement

There is an example Requirement that comes with the plugin that is a “Multi-Requirement”. This groups several Requirements together and supplies an operator, either AND or OR. This is a good way to group Requirements and allows for the OR operator if only one of a certain group of Requirements needs to be true for the Action to be possible.

## Code Factor

To create a Factor in code, go to the code files and create a new class that inherits from *UABUtilityFactorBase*. Name it something that reflects what the Factor will score against.

Override the *CalculateFactorScore()* function and set it up so it calculates a normalised score. Custom functions and members can be added to ensure code cleanliness and customisability, and members can be validated and debugged in the *ValidateFactor()* and *DescribeSelfToGameplayDebugger()* function overrides respectively.

## Blueprint Factor

Factors can also be made in Blueprints. Create a new Blueprint class of type *UABUtilityFactorBlueprintBase* and, in the new Blueprint, override *CalculateBlueprintFactorScore()* and set up the logic that will calculate and return the Factor score. Custom members can be validated and debugged in the overridable functions *ValidateBlueprintFactor()* and *DescribeBlueprintSelfToGameplayDebugger()*.

## Requirement and Factor Setup in Data Asset

Back in the data asset, the Action added previously has two lists, one named Requirements and one named Factors. Add the newly created Requirement and Factor into these lists and set up the correct data for any exposed members.

## Asynchronous Logic and Parallel For

The plugin comes with built-in asynchronous logic and usage of the engines *AsyncTask* and *ParallelFor* helpers. These are not used automatically, as they may not always help optimise the game due to the added overhead.

To enable the asynchronous logic, go to the component set up in the AI Controller Blueprint and enable *RunComponentAsynchronously*. This will use *AsyncTask* to check the Requirements for each Action before choosing an Action once complete.

To use the *ParallelFor* helper in the component, enable *UseParallelRequirementChecking*. This will swap the generic *for* loop for the *ParallelFor* helper and spread out the iterations across threads.

Each Action also has this option, so it can be enabled for individual Actions if that Action should benefit from it.

On the component, both *RunComponentAsynchronously* and *UseParallelRequirementChecking* can be enabled at the same time, but it is not required.

**Ensure to measure performance both before and after enabling or disabling each of these, including after enabling or disabling**

***UseParallelRequirementChecking* on each Action individually to make sure the game is running as best as it can.**

## Validation

When saving the asset, all of the Actions and their Requirements and Factors will automatically be validated. The *UABUtilityDataAssetValidator* class makes sure it's validating a *UABUtilityDataAsset*, and then runs *ValidateAction()* on each of the data assets Actions, *ValidateRequirement()* on each of each Actions Requirements, and *ValidateFactor()* on each of each Actions Factors. It is wise to override the relevant validation function on each Action, Requirement and Factor to ensure that each member of each class is valid at editor time.

## Debugging

The Gameplay Debugger Menu can be enabled at runtime by pressing the apostrophe (') key. One of the categories that should come up will be labelled "UtilityAI". Enabling this category will run *DescribeSelfToGameplayDebugger()* on the Utility Component of the debugged actor and all Actions, Requirements and Factors found within that component, displaying text describing the current values of class members and drawing any debug shapes relating to the current Action being performed. It is wise to override the debugging function on each Action, Requirement and Factor to ensure each member of each class can be debugged at runtime.

## Comments

The plugin is built and maintained by Axela Brockett. Any comments regarding improvements, bugs found or anything else can be directed to her either in the GitHub repository comments or on LinkedIn at

<https://www.linkedin.com/in/axelajimbrockett/>.